

FLOWLY

Application de Productivité Gamifiée

NORMES & MÉTHODOLOGIE

Guide de Développement du Projet

Version 1.0 — Mars 2026

Table des Matières

1. Normes de Code et Conventions	3
2. Definition of Done (DoD)	6
3. Justification de la Méthodologie Agile Scrum	9
4. Organisation des Sprints etRéunions	11
5. Outils et Environnement deTravail	14
6. Comptes-Rendus et Documentation	16

1. Normes de Code et Conventions

1.1 Conventions de Nommage

Élément	Convention	Exemple ✓	Exemple ✗
Variables	camelCase	userScore, isActive	user_score, UserScore
Constantes	SCREAMING_SNAKE	MAX_RETRY, API_URL	maxRetry, apiUrl
Fonctions	camelCase + verbe	getUserData()	userData()
Classes/Components	PascalCase	UserProfile, FocusTimer	userProfile
Fichiers composants	PascalCase.tsx	UserCard.tsx	user-card.tsx
Fichiers utils	kebab-case.ts	date-utils.ts	DateUtils.ts
CSS Classes	kebab-case	focus-timer-btn	focusTimerBtn
Types/Interfaces	PascalCase + I/T	IUser, TSessionData	user, sessionData

1.2 Indentation et Formatage

- Indentation : 2 espaces (pas de tabulations)
- Longueur de ligne maximale : 100 caractères
- Point-virgule obligatoire en fin de ligne
- Guillemets simples (') pour les strings, doubles pour JSX
- Fichiers limités à 300 lignes max, fonctions à 50 lignes max

1.3 Organisation des Imports

Les imports doivent être organisés dans cet ordre, séparés par une ligne vide :

```
//1. React/Next.js
import React, { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';

// 2. Librairies tierces
import { motion } from 'framer-motion';
import * as THREE from 'three';

// 3. Composants internes
import { Button } from '@components/ui/Button';
import { FocusTimer } from '@components/features/FocusTimer';

// 4. Hooks personnalisés
import { useAuth } from '@hooks/useAuth';

// 5. Types/Interfaces
import type { IUser, TSession } from '@types';
```

```
// 6. Styles import styles from
'./Component.module.css';
```

1.4 Configuration ESLint

```
// .eslintrc.js
module.exports = {
  extends: ['next/core-web-vitals', 'prettier'],
  rules: {
    'indent': ['error', 2],
    'quotes': ['error', 'single'],
    'semi': ['error', 'always'],
    'max-len': ['warn', { code: 100 }],
    'no-console': 'warn',
    '@typescript-eslint/explicit-function-return-type': 'error'
  }
};
```

1.5 Configuration Prettier

```
// .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100
}
```

Pour le long terme, nous voulons assurer la maintenabilité et la lisibilité du code.

L'utilisation de formats cohérents que nous avons définis permet à chaque membre de l'équipe de comprendre le contexte et de reconnaître ce qu'un morceau de code représente.

- camelCase → variables et fonctions
- SCREAMING_SNAKE_CASE → constantes
- PascalCase → classes et composants React
- kebab-case → fichiers utilitaires et classes CSS

Cette distinction visuelle réduit l'ambiguïté. Lors de la lecture du code, un développeur peut immédiatement comprendre s'il s'agit d'une variable, d'une constante, d'une fonction ou d'un composant.

ESLint agit comme un système automatisé de contrôle qualité où il nous permet d'imposer des standards de codage et de détecter des problèmes potentiels avant que le code ne soit exécuté.

Prettier est un formateur de code opinionated. Il supprime tout style original et garantit que tout le code généré respecte un style cohérent. Par conséquent, il est dans notre intérêt de maintenir un format uniforme afin de rendre le code plus lisible entre les membres de l'équipe, ce que ces deux technologies nous permettent de faire.

2. Definition of Done (DoD)

2.1 Critères de Validation Obligatoires

Tout ticket doit remplir ces critères pour être considéré comme terminé :

✓	Critère	Validation par
☐	Le code compile sans erreur	CI/CD Pipeline
☐	Aucune erreur ESLint/Prettier	Pre-commit hook
☐	Tests unitaires écrits et passants	Jest/Vitest
☐	Code review approuvée (min. 1 reviewer)	GitHub PR
☐	Documentation mise à jour si nécessaire	Reviewer
☐	Pas de console.log ou code de debug	ESLint rule
☐	Critères d'acceptation de la User Story validés	QA / PO

2.2 Format des User Stories

Chaque User Story suit le format standard :

EN TANT QUE [type d'utilisateur]
JE VEUX [fonctionnalité]
AFIN DE [bénéfice/valeur]

Exemple Concret

US-042: Timer Pomodoro avec détection de distraction

EN TANT QUE utilisateur souhaitant me concentrer.
JE VEUX lancer un timer Pomodoro qui détecte si je quitte l'app de travail.
AFIN DE être alerté de mes distractions et améliorer ma concentration.

Critères d'acceptation :

- Le timer démarre avec la durée configurée (défaut 25min).
- La détection de fenêtre active fonctionne sur Windows/Mac.
- Une notification apparaît après 10s hors focus.
- Les distractions sont loguées avec timestamp.
- Points attribués proportionnellement au temps focus.

2.3 Couverture de Tests Attendue

Type de Test	Couverture Min	Couverture Cible	Outil
Tests Unitaires	70%	85%	Jest / Vitest
Tests Intégration API	80%	95%	Supertest
Tests E2E	Parcours critiques	100% parcours	Playwright

Parcours E2E Critiques

- Inscription et onboarding complet.
- Lancement et complétion d'une session Pomodoro.
- Achat et placement d'un objet 3D.
- Création et participation à une session de groupe.
- Synchronisation desktop-mobile.

Chaque critère a été choisi afin d'assurer la qualité technique et la maintenabilité.

Notre Définition of Done garantit qu'une fonctionnalité n'est pas seulement codée, mais testée, revue et documentée avant d'être mergée.

Nous avons défini des seuils minimum et des objectifs de couverture de tests afin d'assurer la fiabilité et d'éviter les régressions. Cela garantit que la logique métier principale et les parcours utilisateurs critiques sont systématiquement validés. Il n'est pas dans notre intérêt d'atteindre 100 % de couverture partout, car forcer 100 % peut conduire à des tests sans réelle valeur.

Nous avons fixé 70 % comme base non négociable afin d'éviter un code fragile et 85 % comme objectif pour augmenter la fiabilité des tests unitaires.

Pour les tests d'intégration, une couverture de 80 – 95 % est une meilleure solution puisque l'accent est mis sur les endpoints API, l'authentification, le suivi des sessions, les points/transactions et la logique de synchronisation, ce qui est particulièrement important et nécessite une couverture solide.

Nous avons fixé la couverture E2E à 100 % pour les parcours utilisateurs critiques, car ces flux représentent la valeur principale du produit, et garantir leur bon fonctionnement protège la fiabilité globale.

3. Justification de la Méthodologie

3.1 Choix : Agile Scrum

Après analyse des méthodologies disponibles, nous avons choisi Scrum pour le projet Flowly.

3.2 Comparaison des Méthodologies

Critère	Scrum ✓	Kanban	Waterfall
Flexibilité	Haute	Très haute	Faible
Prévisibilité	Bonne (vélocité)	Variable	Haute
Feedback	Toutes les 2 sem.	Continu	Fin de projet
Équipe idéale	5-9 personnes	Flexible	Grande équipe

3.3 Pourquoi Scrum pour Flowly ?

Adéquation avec le projet

- Nature exploratoire : Le monde 3D et la gamification nécessitent des itérations rapides.
- Équipe de taille moyenne : Notre équipe de 6-8 personnes correspond au cadre Scrum.
- Feedback régulier : Les beta-testeurs donnent leur avis à chaque fin de sprint.
- Évolution des priorités : Backlog repriorisable selon les retours marché.

Avantages spécifiques

- Livraisons fréquentes : Version déployable toutes les 2 semaines.
- Visibilité : Le Product Owner suit l'avancement en temps réel.
- Qualité intégrée : Tests et documentation inclus dans chaque print.
- Amélioration continue : Rétrospectives pour optimiser les processus.
- Gestion des risques : Identification précoce des blocages (notamment 3D).

Waterfall

La méthodologie Waterfall est un workflow de gestion de projet bien établi.

Le projet est divisé en phases fixes :

- Requirements
- Design
- Development
- Testing
- Deployment

Chaque phase doit être complétée avant de passer à la suivante.

La méthodologie Waterfall offre un cadre de projet clair et structuré avec des objectifs définis, des coûts et des délais prévisibles. Elle permet un suivi plus simple, des responsabilités claires et fonctionne bien pour des projets avec des exigences fixes et clairement définies. Cependant, sa rigidité pose certaines contraintes dans le cadre du projet. Elle est plus adaptée à des exigences très stables et bien définies.

Exemple de scénario : Développer un système de reporting bancaire de conformité requis par la loi. Pour ce projet, les exigences sont clairement définies et les changements sont minimes.

Agile

Agile est une approche flexible de gestion de projet.

Au lieu de tout planifier dès le début, le projet est développé étape par étape en petites parties.

Elle permet à l'équipe de :

- S'adapter aux changements
- Obtenir des retours réguliers
- Améliorer le projet en continu

Agile est utile pour des projets évolutifs où les exigences peuvent changer au fil du temps.

Scrum

Scrum est un framework structuré au sein d'Agile. Il divise le projet en cycles courts appelés sprints (généralement deux semaines).

À la fin de chaque sprint, l'équipe livre une partie fonctionnelle du produit.

Cela crée un rythme clair et aide l'équipe à rester organisée.

Exemple de scénario : Construire une application mobile de startup.

Pourquoi Scrum fonctionne : les exigences évoluent, les utilisateurs donnent fréquemment du feedback, les fonctionnalités doivent être itérées et l'équipe collabore étroitement.

Kanban

Kanban est un système de workflow flexible où les tâches passent continuellement de “To Do” à “In Progress” puis à “Done”. Il n’y a pas de sprints fixes. Cependant, Kanban ne fournit pas une structure temporelle forte. Il ne propose pas d’objectifs de sprint clairs, de cycles de livraison définis ni de progression mesurable.

Exemple de scénario : Gérer une équipe de support client.

Les tâches arrivent de manière imprévisible, il n’y a pas de durée de sprint fixe et les priorités changent quotidiennement. Un flux continu est donc nécessaire.

Au final, nous avons choisi Scrum car il nous apporte une structure grâce aux sprints et aux revues régulières, tout en conservant de la flexibilité. Waterfall était trop rigide et Kanban trop peu structuré pour les contraintes de notre projet.

Organisation des sprints

Nous avons choisi des sprints de 2 semaines car ils offrent un bon équilibre entre le temps de développement et les retours fréquents. Ils sont suffisamment longs pour livrer des fonctionnalités significatives, mais suffisamment courts pour s’adapter rapidement aux changements ou aux retours.

4. Organisation des Sprints et Réunions

4.1 Cadence des Sprints

Paramètre	Valeur
Durée du Sprint	2 semaines (10 jours ouvrés)
Début du Sprint	Lundi matin (après Sprint Planning)
Fin du Sprint	Vendredi (Sprint Review + Rétro)

4.2 Cérémonies Scrum

Cérémonie	Durée	Fréquence	Objectif
Daily Stand-up	15 min	Quotidien 9h30	Synchronisation équipe, blocages
Sprint Planning	2-3h	Lundi J1	Sprint Goal, sélection US, estimations
Sprint Review	1h	Vendredi J10	Démo, feedback stakeholders
Rétrospective	1h	Vendredi J10	Amélioration continue (Start/Stop/Continue)
Refinement	1h	Mercredi	Affiner US, critères d'acceptation

4.3 Calendrier Type d'un Sprint

	Lundi	Mardi	Mercredi	Jeudi	Vendredi
Sem. 1	Planning + Daily	Daily	Daily + Refine	Daily	Daily
Sem. 2	Daily	Daily	Daily + Refine	Daily	Review + Rétro

5. Outils et Environnement de Travail

5.1 Stack d'Outils (Gratuits)

Catégorie	Outil	Usage
Gestion Projet	ClickUp (Free)	Backlog, sprints, tickets, roadmap, docs
Communication	Discord	Channels équipe, vocaux, partage écran
Cérémonies / Visio	Discord	Daily, cérémonies Scrum, pair programming
Code Repository	GitHub	Versioning, PR, code review, CI/CD
Documentation	ClickUp Docs	Wiki technique, specs, CR réunions
Design	Figma (Free)	Maquettes, prototypes, design system

5.2 Organisation Discord

Channels Texte

- #général — Annonces et discussions générales
- #dev — Discussions techniques, questions dev
- #design — UX/UI, reviews design
- #bugs — Rapports de bugs, triage
- #daily-standup — Résumé écrit des daily (async si besoin)
- #liens-utiles — Ressources, documentation, liens importants

Channels Vocaux

- 🗣️ Daily — Pour les stand-ups quotidiens (15 min)
- 🗣️ Cérémonies — Pour Planning, Review, Rétro, Refinement
- 🗣️ Pair Programming — Sessions de code en binôme
- 🗣️ Chill — Discussions informelles, pause café virtuelle

5.3 Workflow Git

Stratégie de Branching

```
main (production)
├── develop (intégration)
│   ├── feature/FLOW-123-timer-pomodoro
│   ├── bugfix/FLOW-125-session-sync
│   └── hotfix/FLOW-126-critical-fix
```

Convention de Nommage des Branches

- feature/FLOW-[ticket]-description-courte
- bugfix/FLOW-[ticket]-description-courte
- hotfix/FLOW-[ticket]-description-courte

Convention de Commit (Conventional Commits)

```
feat(timer): add pomodoro detection feature
fix(auth): resolve OAuth token refresh issue
docs(readme): update installation instructions
refactor(3d): optimize world rendering
test(api): add integration tests for focus service
```

6. Comptes-Rendus et Documentation

6.1 Types de Documentation

Document	Fréquence	Responsable	Stockage
CR Daily Stand-up	Quotidien	Scrum Master	Discord #daily-standup
CR Sprint Planning	Chaque sprint	Scrum Master	ClickUp Docs ClickUp
CR Sprint Review	Chaque sprint	Product Owner	Docs ClickUp Docs
CR Rétrospective	Chaque sprint	Scrum Master	(privé)

6.2 Template CR Sprint Planning

SPRINT PLANNING — Sprint [N]

Date : [DATE] | Participants : [LISTE]

Sprint Goal : [Objectif en 1-2 phrases]

User Stories sélectionnées :

- US-XXX : [Titre] — [X] points
- US-XXX : [Titre] — [X] points

Capacité : [X] pts | Engagement : [Y] pts

Risques : [Risques identifiés]

6.3 Template CR Rétrospective (Start/Stop/Continue)

✓ START À commencer	✗ STOP À arrêter	→ CONTINUE À continuer
• [Item 1] • [Item 2] • [Item 3]	• [Item 1] • [Item 2] • [Item 3]	• [Item 1] • [Item 2] • [Item 3]

Actions retenues :

10. [Action 1] —Responsable : [Nom] — Deadline : [Date]

11. [Action 2] —Responsable : [Nom] — Deadline : [Date]

— **Fin du Document** —

Flowly — Normes & Méthodologie v1.0